Объект хандалтат программчлалын удамшлын хэрэглээ

Лабораторийн ажил 8

МУИС, МТЭС, МКУТ-ийн суралцагч

Э. Баттамир, 24B1NUM0616@stud.num.edu.mn

А. Баяржавхлан, 24B1NUM0876@stud.num.edu.mn

Б. Маргад-Эрдэнэ, 24B1NUM1418@stud.num.edu.mn

8.1 Оршил

Энэхүү лабораторийн ажлын хүрээнд Shape классыг удамшил ашиглан хоёр хэмжээст дүрсүүдийг (тойрог, дөрвөлжин, гурвалжин) загварчилж, полиморфизм, хийсвэр класс, жинхэнэ хийсвэр функц, static гишүүд, this заагч зэрэг ойлголтуудыг онолын хүрээнд практик байдлаар хэрэгжүүлнэ.

Мөн дүрсүүдийг динамик заагч хүснэгтэд хадгалж талбай болон периметрээр эрэмбэлэх алгоритм боловсруулж, хийсвэр функц ашиглан уян хатан систем бүтээнэ.

8.2 Зорилго

1. Хийсвэр класс болон жинхэнэ хийсвэр функцийн хэрэглээг ойлгож, хэрэгжүүлэх
2. Полиморфизмын зарчмыг ашиглан өөр өөр төрлийн дүрсүүдийг нэг интерфэйсээр удирдах
3. static гишүүн өгөгдөл болон функцийн хэрэглээг ойлгож, объектуудыг тоолох систем хэрэгжүүлэх
4. this заагчийг ашиглан классын дотоод өгөгдөл болон функцэд хандах аргыг дадлагажуулах
5. Хоёр хэмжээст дүрсүүдийн талбай болон периметрийг тооцоолж, жинхэнэ хийсвэр функц ашиглан удам классууд дээр дахин тодорхойлох
6. Дүрсүүдийг талбай болон периметрийн утгаар нь эрэмбэлэх алгоритм хэрэгжүүлэх
7. Класс хоорондын удамшил болон хийсвэрлэлийг ашиглан өргөтгөх боломжтой систем зохиох чадвар эзэмших

8.3 Онолын судалгаа

1. **This хувьсагч гэж юу вэ?**

   this хувьсагч нь C++ хэл дээрх тусгай зориулалтын заагч бөгөөд гишүүн функцийг дуудаж байгаа объектын хаягийг заадаг. C++ компайлер нь гишүүн функц дуудагдах үед тухайн функцийг дуудаж буй объектын хаягийг автоматаар this хувьсагчид оноож, функцийг ажиллуулахдаа далд параметр хэлбэрээр дамжуулдаг. Өөрөөр хэлбэл, this нь программист өөрөө зарлахгүй ч автоматаар ашиглагддаг дотоод хувьсагч юм.
   this хувьсагчийг гишүүн функц дотор далд (implicit) болон ил (explicit) хоёр хэлбэрээр ашиглаж болно. Далд хэрэглээнд гишүүн өгөгдөлд шууд ханддаг бол, ил хэрэглээнд this-> гэсэн хэлбэрийг ашиглана.

2. **Полиморфизм гэж юу вэ? Хэдэн янз байдаг вэ?**

   Полиморфизм гэдэг нь объект хандалтат програмчлалд нэг ижил нэртэй функц өөр өөр нөхцөлд өөр өөрөөр ажиллах боломжийг хэлнэ. Өөрөөр хэлбэл, нэг интерфэйс ашиглан олон төрлийн объект дээр өөр өөр үйлдэл гүйцэтгэх чадвар юм.

   Полиморфизм нь 3 төрөлтэй:

   1) **Функц дахин тодорхойлох**
      Нэг классын дотор ижил нэртэй, өөр параметртэй олон функц зарлах.
   2) **Функц дахин программчлах**
      Суурь класст зарлагдсан virtual функцийг удамшсан класст дахин хэрэгжүүлэхийг хэлнэ. Аль функц дуудагдах нь програм ажиллах үед шийдэгднэ.
   3) **Оператор дахин тодорхойлох**
      C++ хэл дээрх операторууд (+, -, *, <<, == гэх мэт)-ын үйлдлийг хэрэглэгчийн тодорхойлсон класст гишүүн функцаар тохируулан дахин тодорхойлохыг хэлнэ.

3. **Static гишүүн өгөгдөл гэж юу вэ? Static гишүүн өгөгдлийн амьдралын мөчлөг ямар байдаг вэ?**

   Static гишүүн өгөгдөл нь классын бүх объектод нийтлэг (shared) байдлаар ашиглагддаг хувьсагч юм. Өөрөөр хэлбэл, объект бүрт тусдаа биш, харин нэг л хувь санах ойд хадгалагдана.

   Амьдралын мөчлөг:

   - Програм эхлэх үед нэг удаа үүснэ
   - Бүх объектууд ашиглана
   - Програм дуусах хүртэл оршдог

   Иймээс static хувьсагч нь объект үүссэн, устсан эсэхээс үл хамааран хадгалагддаг.

4. **Static гишүүн өгөгдөлд хэрхэн хандах вэ? Объектоор эсвэл классаар дамжуулж хандах.**
   1) **Объектоор дамжуулж хандах**
      ```
      Shape c; c.count;
      ```
   2) **Классаар дамжуулж хандах**
      ```
      Shape::count;
      ```
5. **Static гишүүн функцийг хэрхэн зарладаг, дууддаг вэ?**
   Static гишүүн функцийг static түлхүүр үгээр зарлана.
   ```
   static int getCount();
   ```
   Static гишүүн функцийг объект үүсгэхгүй классын нэрийг ашиглан дуудна.
   ```
   return Shape::getCount();
   ```

6. **Жинхэнэ хийсвэр функц гэж юу вэ?**
   Хийсвэр функц нь классын нэг хэсэг бөгөөл тухайн классын зарим функц нь зөвхөн тодорхойлолттой зарлагдсан бөгөөд хэрэгжүүлэлтгүй байдаг. Хийсвэр функцтэй классыг хийсвэр класс гэж нэрлэдэг.
   Хийсвэр функц нь virtual түлхүүр үгээр зарлагдаж, тэнцүү тэг гэж бичигддэг. Энэ нь тухайн функцын хэрэгжүүлэлтийг удамшсан ангиудад зааавал тодорхойлох ёстойг илэрхийлнэ.

7. **Хийсвэр класс гэж юу вэ? Хэрхэн ашигладаг вэ?**
   Хийсвэр класс нь нэг эсвэл түүнээс олон хийсвэр функц агуулсан класс юм. Энэ нь загварчлалын зориулалттай бөгөөд дангаараа объект үүсгэж болдоггүй. Хийсвэр класс нь программд ерөнхий зан үйлийг тодорхойлж, удамшсан класст тусгайлсан хэрэгжүүлэлтийг заавал хийх шаардлага тавьдаг.

   Удамшсан классууд эх классын хийсвэр функцийг заавал override хийж, өөрийн гэсэн үйлдлийг бичнэ. Ингэснээр хийсвэр байсан функц бодит болж функц болж дахин тодорхойлогдоно.

8. **Жинхэнэ хийсвэр функцийн давуу тал.**
   1) **Стандарт интерфэйс бий болгоно**
      Бүх удам классууд ижил функцтэй байхыг албадна
   2) **Полиморфизм хэрэгжүүлэх боломж олгоно**
      Нэг pointer-оор олон төрлийн объект ажиллуулна
   3) **Кодын уян хатан байдлыг нэмэгдүүлнэ**
      Шинэ класс нэмэхэд хуучин код өөрчлөх шаардлагагүй
   4) **Алдаа багасгана**
      Хэрэгжүүлэх ёстой функцыг мартах боломжгүй

8.4 Хэрэгжүүлэлт

1. Үүссэн объектийг тоолох статик хувьсагч

```
class Shape {
    protected:
        string name;
        static int count;

int Shape::count = 0;

Shape::Shape() : name("Shape") { Shape::count++; }
Shape::Shape(const string& name) : name(name) { Shape::count++; }

#include "../oop.h"

int obj_count() { return Shape::getCount(); }
```

2. this хувьсагч ашиглах

```
Circle::Circle(double radius, double crd_1, double crd_2) :
Shape2D("Circle") {
    this->radius = radius;
    this->crd_1 = crd_1;
    this->crd_2 = crd_2;
}
```

3. Shape2D класс төрөлтэй объектуудыг заах заагч хүснэгт

```
Shape2D **shapes = new Shape2D*[cap];

while (true) {
        cout << "1. Make circle   2. Make square
                 3. Maketriangle   4. Exit" << endl;
```

4. Заагч хүснэгтийн хэмжээг динамикаар удирдах.

```
Shape2D **newArr = new Shape2D*[cap];

for (int i = 0; i < siz; i++) {
 newArr[i] = shapes[i];
}

delete[] shapes;
shapes = newArr;
```

5. Shape2D төрөлтэй объектуудыг талбайгаар эрэмбэлэх.

```
void sortArea(Shape2D* list[]) {
    cout << endl;
    for (int i = 1; i < obj_count(); i++) {
        Shape2D* key = list[i];
        int j = i - 1;
```

```
        while (j >= 0 && list[j]->area() > key->area()) {
            list[j + 1] = list[j];
            j--;
        }
        list[j + 1] = key;
    }
    cout << "Sorted by Area..." << endl;
}
```

6. main() функцад ашиглах.

```
sortArea(shapes);

print(shapes);

sortPerimeter(shapes);

print(shapes);

cout << endl << obj_count() << " objects created." << endl;
```

8.5 Багийн ажиллагаа

1. **Гишүүдийн оролцоо**
   Э. Баттамир – **полиморфизм**
   А. Баяржавхлан – **статик хувьсагч ба гишүүн функц**
   Б. Маргад-Эрдэнэ – **this хувьсагч**

8.6 Дүгнэлт

Энэхүү лабораторийн ажлын хүрээнд объект хандалтат програмчлалын удамшил сэдвийн үндсэн ойлголтууд болох this заагч, polymorphism, static гишүүн өгөгдөл ба функц, жинхэнэ хийсвэр функц, хийсвэр класс, мөн operator overloading зэргийг онол болон практик байдлаар онолын хүрээнд судалж хэрэгжүүллээ.

Полиморфизмын тусламжтайгаар нэг интерфэйсээр олон төрлийн объектуудыг удирдах боломж бүрдэж, кодын уян хатан байдал нэмэгдсэн. Мөн жинхэнэ хийсвэр функц болон хийсвэр классыг ашигласнаар удамшсан классуудын заавал хэрэгжүүлэх ёстой стандарт бүтэц тодорхойлогдож, системийн өргөтгөх боломжийг сайжруулан объект хандалтат програмчлалын үндсэн зарчмуудыг практик даалгаварт амжилттай хэрэгжүүлж, програмын бүтэц, логикийг илүү цэгцтэй, дахин ашиглах боломжтой байдлаар зохион байгуулах чадвар эзэмшлээ.

8.7 Ашигласан материал

1. М. Золжаргал багшийн лекцийн материал
2. Объект хандлагат технологийн С++ программчал, Ж. Пүрэв, 2008, Улаанбаатар
3. geeksforgeeks

8.8 Хавсралт

1. oop.h

```
#ifndef OOP_H
#define OOP_H

#include <string>
#include <iostream>
#include <cmath>
using namespace std;

class Shape {
    protected:
        string name;
        static int count;
    public:
        Shape();
        Shape(const string& name);
        void setName(const string& name);
        string getName() const;
        static int getCount();
};

class Shape2D : public Shape {
    public:
        Shape2D() : Shape() {};
        Shape2D(string name) : Shape(name) {};
        virtual double area() const = 0;
        virtual double perimeter() const = 0;
};

class Circle : public Shape2D {
    private:
        double crd_1;
        double crd_2;
        double radius;
    public:
        Circle();
        Circle(double radius, double crd_1, double crd_2);
```

```
        double area() const override;
        double perimeter() const override;
        void showCoordinate();
        void setRadius(double n);
        void setX(double crd_1);
        void setY(double crd_2);
        double getRadius() const;
        double getX() const;
        double getY() const;
};

class Square : public Shape2D {
    private:
        double x0, y0;
        double x1, y1;
        double x2, y2;
        double x3, y3;
        double side;
    public:
        Square();
        Square(double side, double x0, double y0);
        double area() const override;
        double perimeter() const override;
        void ShowCoordinates() const;
        void setSide(const double side);
        void setCoordinates(double x0, double y0);
        double getSide() const;
};

class Triangle : public Shape2D {
    private:
        double x0, y0;
        double x1, y1;
        double x2, y2;
        double side;
    public:
        Triangle();
        Triangle(double side, double x0, double y0);
        double area() const override;
        double perimeter() const override;
        void showCoordinates() const;
        void setSide(double n);
        void setCoordinates(double x0, double y0);
        double getSide();
};

int obj_count();
```

```
void sortArea(Shape2D* list[]);
void sortPerimeter(Shape2D* list[]);
void print(Shape2D** shapes);

#endif
```

2. circle.cpp

```
#include "../oop.h"

Circle::Circle() : Shape2D("Circle") {
    this->radius = 0;
    this->crd_1 = 0;
    this->crd_2 = 0;
}
Circle::Circle(double radius, double crd_1, double crd_2) : Shape2D("Circle") {
    this->radius = radius;
    this->crd_1 = crd_1;
    this->crd_2 = crd_2;
}

void Circle::setRadius(double radius) {
    this->radius = radius;
}

void Circle::setX(double crd_1) {
    this->crd_1 = crd_1;
}

void Circle::setY(const double crd_2) {
    this->crd_2 = crd_2;
}

double Circle::getRadius() const{
    return this->radius;
}

double Circle::area() const {
    return M_PI * this->radius * this->radius;
}

double Circle::perimeter() const {
    return 2 * M_PI * this->radius;
}

void Circle::showCoordinate() {
```

```
    cout << "Center: (" << this->crd_1 << ", " << this->crd_2 << ")" << endl;
}

double Circle::getX() const{
    return this->crd_1;
}

double Circle::getY() const{
    return this->crd_2;
}
```

3. Max.cpp

```
#include "../oop.h"

int max(double *areas, int n) {
    int maxidx = 0;
    for (int i = 1; i < n; i++ ) {
        if (*(areas + i) > *(areas + maxidx)) maxidx = i;
    }

    return maxidx;
}
```

4. shape.cpp

```
#include "../oop.h"

int Shape::count = 0;

Shape::Shape() : name("Shape") { Shape::count++; }
Shape::Shape(const string& name) : name(name) { Shape::count++; }

void Shape::setName(const string& name) {
    this->name = name;
}

string Shape::getName() const {
    return name;
}

int Shape::getCount() {
    return count;
}
```

5. square.cpp

```
#include "../oop.h"

Square::Square() : Shape2D("Square") {
	this->side = 0;

	this->x0 = 0;
	this->y0 = 0;

	this->x1 = 0;
	this->y1 = 0;

	this->x2 = 0;
	this->y2 = 0;

	this->x3 = 0;
	this->y3 = 0;
}

Square::Square( double side, double x0, double y0) : Shape2D("Square") {
	this->side = side;

	this->x0 = x0;
	this->y0 = y0;

	this->x1 = this->x0 + this->side;
	this->y1 = this->y0;

	this->x2 = this->x1;
	this->y2 = this->y1 + this->side;

	this->x3 = this->x0;
	this->y3 = this->y0 + this->side;
}

void Square::setSide(double side) {
	this->side = side;
}

void Square::setCoordinates(double x0, double y0) {
	this->x0 = x0;
	this->y0 = y0;

	this->x1 = this->x0 + this->side;
	this->y1 = this->y0;
```

```
        this->x2 = this->x1;
        this->y2 = this->y1 + this->side;

        this->x3 = this->x0;
        this->y3 = this->y0 + this->side;
}

double Square::getSide() const {
        return this->side;
}

double Square::area() const {
        return this->side * this->side;
}

double Square::perimeter() const {
        return 4 * this->side;
}

void Square::ShowCoordinates() const {

        cout << "Coordinates:" << endl;

        cout << "Left Bottom (" << this->x0 << ", " << this->y0 << ")" << endl;
        cout << "Right Bottom (" << this->x1 << ", " << this->y1 << ")" << endl;
        cout << "Right Top (" << this->x2 << ", " << this->y2 << ")" << endl;
        cout << "Left Top (" << this->x3 << ", " << this->y3 << ")" << endl;
}
```

6. triangle.cpp

```
#include "../oop.h"

Triangle::Triangle() : Shape2D("Triangle") {
    this->side = 0;

    this->x0 = 0;
    this->y0 = 0;
    this->x1 = 0;
    this->y1 = 0;
    this->x2 = 0;
    this->y2 = 0;
}
```

```
Triangle::Triangle(double side, double x0, double y0) : Shape2D("Triangle") {
    this->side = side;
    double height = (sqrt(3) / 2) * side;

    this->x0 = x0;
    this->y0 = y0;

    this->x1 = this->x0 - this->side / 2;
    this->y1 = this->y0 - height;

    this->x2 = this->x0 + this->side / 2;
    this->y2 = this->y0 - height;
}

void Triangle::setSide(double side) {
    this->side = side;
}

void Triangle::setCoordinates(double x0, double y0) {
    this->x0 = x0;
    this->y0 = y0;

    this->x1 = this->x0 - this->side / 2;
    this->y1 = this->y0 - (sqrt(3) / 2) * this->side;

    this->x2 = this->x0 + side / 2;
    this->y2 = this->y0 - (sqrt(3) / 2) * this->side;
}

double Triangle::getSide() {
    return this->side;
}

double Triangle::area() const {
    return (sqrt(3)/4) * this->side * this->side;
}

double Triangle::perimeter() const {
    return 3 * this->side;
}

void Triangle::showCoordinates() const {

    cout << "Coordinates:" << endl;
```

```
    cout << "A(" << x0 << ", " << y0 << ")" << endl;
    cout << "B(" << x1 << ", " << y1 << ")" << endl;
    cout << "C(" << x2 << ", " << y2 << ")" << endl;
}
```

7. main.cpp

```
#include "oop.h"

int main () {
        int k;
        int cap = 2;
        int siz = 0;

        Shape2D **shapes = new Shape2D*[cap];

        while (true) {
                cout << "1. Make circle 2. Make square 3. Make triangle 4. Exit" << endl;
                cin >> k;
                double side, radius;
                if (k == 4) break;

                if (siz == cap) {
                        cap *= 2;

                        Shape2D **newArr = new Shape2D*[cap];

                        for (int i = 0; i < siz; i++) {
                                newArr[i] = shapes[i];
                        }

                        delete[] shapes;
                        shapes = newArr;
                }

                switch (k)
                {
                        case 1:
                                cout << "Radius: ";
                                cin >> radius;
                                shapes[siz++] = new Circle(radius, 0, 0);
                                break;
                        case 2:
                                cout << "Side: ";
                                cin >> side;
                                shapes[siz++] = new Square(side, 0, 0);
                                break;
```

```
                        case 3:
                                cout << "Side: ";
                                cin >> side;
                                shapes[siz++] = new Triangle(side, 0, 0);
                                break;
                        default:
                                cout << "Invalid choice" << endl;
                                continue;
                }
        }

        sortArea(shapes);
        print(shapes);

        sortPerimeter(shapes);

        print(shapes);

        cout << endl << obj_count() << " objects created." << endl;

        for (int i = 0; i < n; i++) {
                delete shapes[i];
        }
        delete[] shapes;

        return 0;
}
```

8. MakeFile

```
all: go

circle.o: classes/circle.cpp oop.h
        g++ -c classes/circle.cpp -o circle.o

shape.o: classes/shape.cpp oop.h
        g++ -c classes/shape.cpp -o shape.o

shape2d.o: classes/shape2d.cpp oop.h
        g++ -c classes/shape2d.cpp -o shape2d.o

square.o: classes/square.cpp oop.h
        g++ -c classes/square.cpp -o square.o
```

```
triangle.o: classes/triangle.cpp oop.h
	g++ -c classes/triangle.cpp -o triangle.o

sort.o: functions/sort.cpp oop.h
	g++ -c functions/sort.cpp -o sort.o

print.o: functions/print.cpp oop.h
	g++ -c functions/print.cpp -o print.o

obj_count.o: functions/count.cpp oop.h
	g++ -c functions/count.cpp -o obj_count.o

main.o: main.cpp oop.h
	g++ -c main.cpp -o main.o

go: circle.o sort.o obj_count.o print.o shape.o shape2d.o square.o triangle.o main.o
		g++ circle.o sort.o obj_count.o print.o shape.o shape2d.o square.o triangle.o
	main.o -o go

clean:
rm -v *.o go
```

Даалгавар байршиж буй холбоос: Github